

Blood Bank Donation and Issue Management System

A Database Management System Project

Student Name	Soubhik Ghosh
Roll Number	25WU0101137
Programme	B.Tech – Computer Science & Engineering (AI & ML)
University	Woxsen University
Faculty	Dr. Kiranmayee Advala
Academic Year	2026–2027

Submitted in partial fulfillment of the requirements for the Database Management Systems course.

2026–2027

Table of Contents

01	Abstract
02	Introduction and Problem Statement
03	Objectives and Scope
04	Software and Hardware Requirements
05	ER Diagram
06	Relational Schema and Normalization
07	Data Dictionary
08	SQL Commands Used (DDL and DML)
09	Queries with Outputs
10	UI Design and Screenshots
11	Implementation Details
12	Testing
13	Conclusion and Future Enhancements
14	References
15	Appendix – GitHub Repository Link

Abstract

The Blood Bank Donation and Issue Management System is a database-driven web application designed to manage the lifecycle of blood donation, storage, requests and issue operations. It provides a centralized platform for maintaining donor, patient, hospital, donation, inventory, request and issue records.

The system uses MySQL as the relational database and Flask/Python as the backend web framework, with HTML, CSS and JavaScript for the user interface. Primary keys, foreign keys, CHECK constraints, UNIQUE constraints, views and database triggers are used to maintain data integrity and automate selected inventory operations.

The project demonstrates practical DBMS concepts including relational modeling, normalization, SQL DDL/DML, joins, aggregation, CRUD operations, views, triggers and application–database connectivity.

1. Introduction and Problem Statement

1.1 Introduction

Blood banks manage sensitive operational information such as donor registrations, blood collection, blood-group classification, inventory availability, patient requirements and hospital requests. When information is maintained manually or in disconnected files, it becomes difficult to maintain consistency and quickly identify available blood stock.

This project implements a centralized relational database with a web interface. Users can manage donors, patients and hospitals, record donations, monitor inventory, create blood requests and record blood issues.

1.2 Problem Statement

The main problem is the absence of a single structured system for managing blood-bank records and stock movement. Manual approaches can cause duplicate records, inconsistent data, slow searching and difficulty in tracking blood movement. The proposed system addresses these issues using relational database design, integrity constraints and a web interface.

1.3 Proposed Solution

A Flask web application is connected to a MySQL database. Separate modules handle donors, patients, hospitals, donations, inventory, requests and blood issues. Database triggers and constraints provide additional protection for critical operations.

2. Objectives and Scope

2.1 Objectives

#	Objective
1	Create a centralized database for blood-bank records.
2	Maintain donor, patient and hospital information.
3	Record blood donations and screening details.
4	Track blood inventory by blood group.
5	Manage hospital/patient blood requests.
6	Record blood issued against requests.
7	Maintain referential integrity using foreign keys.
8	Use triggers to automate inventory/request updates.
9	Provide a user-friendly web interface.

2.2 Scope

The scope covers donor, patient and hospital management; donation recording; inventory monitoring; request processing; issue recording; and database-level integrity. Future extensions can include authentication, role-based access, cloud deployment, notifications, expiry alerts and analytics.

3. Software and Hardware Requirements

3.1 Software Requirements

Component	Requirement
Operating System	Windows 10/11 or equivalent
Programming Language	Python
Web Framework	Flask
Database	MySQL
Database Client	MySQL Workbench
Frontend	HTML5, CSS3, JavaScript
Code Editor	Visual Studio Code
Python Libraries	Flask, mysql-connector-python, python-dotenv
Browser	Chrome / Edge / Firefox
Version Control	Git and GitHub

3.2 Hardware Requirements

Component	Recommended
Processor	Intel Core i3 / Ryzen 3 or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 5 GB free space
Network	Internet connection for deployment
Display	1366×768 or higher

4. ER Diagram

The ER model identifies the major entities and relationships. Donors provide donations; donations are represented in inventory; patients and hospitals participate in requests; and blood issues connect requests with inventory and patient/hospital information.

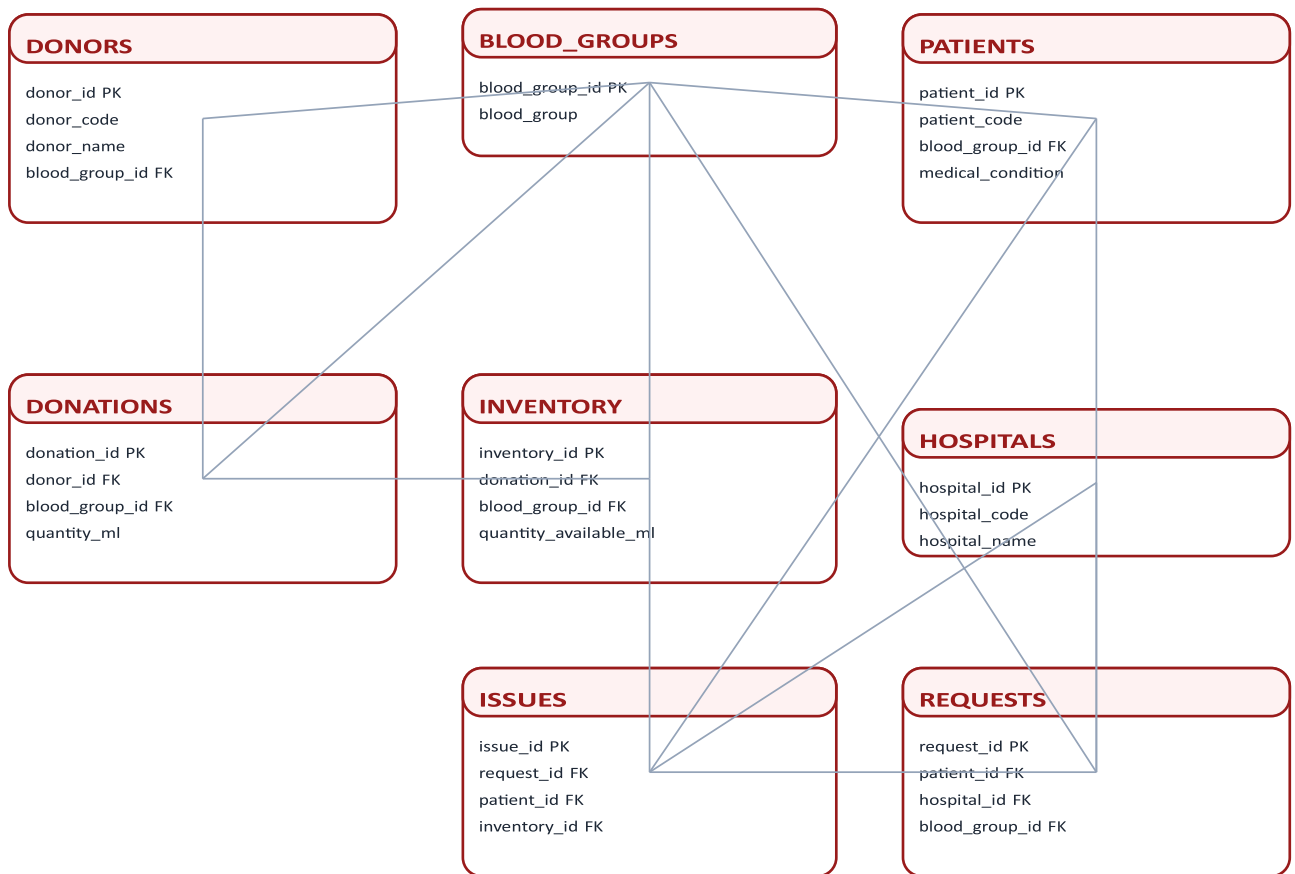


Figure 1. Conceptual ER diagram of the Blood Bank Donation and Issue Management System.

5. Relational Schema and Normalization

5.1 Relational Schema

The database contains eight primary relations: BLOOD_GROUPS, DONORS, HOSPITALS, PATIENTS, DONATIONS, BLOOD_INVENTORY, BLOOD_REQUESTS and BLOOD_ISSUES.

Relation	Primary Key	Important Foreign Keys
blood_groups	blood_group_id	—
donors	donor_id	blood_group_id → blood_groups
hospitals	hospital_id	—
patients	patient_id	blood_group_id → blood_groups
donations	donation_id	donor_id, blood_group_id
blood_inventory	inventory_id	donation_id, blood_group_id
blood_requests	request_id	patient_id, hospital_id, blood_group_id
blood_issues	issue_id	request_id, patient_id, hospital_id, blood_group_id, inventory_

5.2 Normalization

The design follows normalization principles. In 1NF, attributes are atomic and repeating groups are avoided. In 2NF, non-key attributes depend on the complete primary key. In 3NF, non-key attributes do not depend on other non-key attributes. Blood-group names are stored once in BLOOD_GROUPS and referenced using blood_group_id, reducing duplication.

5.3 Views

View	Purpose
donor_details	Donor information with blood-group details.
donation_details	Donation information with donor and blood group.
inventory_details	Inventory records with blood-group information.
request_details	Request, patient, hospital and blood-group information.
blood_issue_details	Blood issue, patient, hospital, inventory and blood-group information.

5.4 Triggers

before_donation_insert: validates that donation quantity is exactly 150 mL. after_blood_issue_insert: reduces inventory quantity and changes status when stock reaches zero; it also marks the associated request as Completed.

6. SQL Commands Used (DDL and DML)

6.1 DDL – Blood Groups

```
CREATE TABLE blood_groups (
    blood_group_id
    INT AUTO_INCREMENT PRIMARY KEY,
    blood_group
    VARCHAR(3) NOT NULL UNIQUE );
```

6.2 DDL – Donors

```
CREATE TABLE donors (
    donor_id INT
    AUTO_INCREMENT PRIMARY KEY,
    donor_code
    VARCHAR(10) NOT NULL UNIQUE,
    donor_name
    VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    gender ENUM('Male','Female','Other') NOT NULL,
    blood_group_id INT NOT NULL,
    phone
    VARCHAR(15) NOT NULL UNIQUE,
    email
    VARCHAR(100) UNIQUE,
    address VARCHAR(200),
    city VARCHAR(50),
    registration_date DATE NOT
    NULL,
    eligibility_status ENUM('Eligible','Not Eligible')
    DEFAULT 'Eligible',
    FOREIGN KEY (blood_group_id)
    REFERENCES blood_groups(blood_group_id),
    CHECK (age >= 18 AND age <= 65)
);
```

6.3 DML – Insert Example

```
INSERT INTO donors (donor_code, donor_name, age,
gender, blood_group_id, phone, email, address,
city, registration_date) VALUES
('D001','Example Donor',21,'Male',1,
'9876543210','example@email.com',
'Hyderabad','Hyderabad','2026-01-15');
```

6.4 Inventory Trigger

```
CREATE TRIGGER after_blood_issue_insert
AFTER INSERT ON blood_issues
FOR EACH ROW
BEGIN
```

```

        UPDATE blood_inventory      SET quantity_available_ml
=          quantity_available_ml -
NEW.quantity_issued_ml,          status = CASE
WHEN quantity_available_ml
NEW.quantity_issued_ml = 0
        THEN 'Issued'
        ELSE 'Available'
        END      WHERE inventory_id =
NEW.inventory_id;

        UPDATE blood_requests
        SET request_status = 'Completed'
        WHERE request_id = NEW.request_id;
END;
```

7. Queries with Outputs

7.1 Count Donors

```
SELECT COUNT(*) AS total_donors FROM donors;
```

Output

50

7.2 Donors with Blood Group

```
SELECT d.donor_code, d.donor_name, bg.blood_group
FROM donors d
JOIN blood_groups bg
ON d.blood_group_id = bg.blood_group_id
ORDER BY d.donor_id;
```

Output: donor records are returned with their corresponding blood-group names.

7.3 Inventory Summary

```
SELECT bg.blood_group,
        SUM(bi.quantity_available_ml) AS total_quantity
FROM blood_groups bg
LEFT JOIN blood_inventory bi
ON bg.blood_group_id = bi.blood_group_id
GROUP BY bg.blood_group_id, bg.blood_group
ORDER BY bg.blood_group_id;
```

7.4 Requests by Hospital

```
SELECT h.hospital_name,
        COUNT(br.request_id) AS request_count
FROM hospitals h
LEFT JOIN blood_requests br
ON h.hospital_id = br.hospital_id
GROUP BY h.hospital_id, h.hospital_name;
```

7.5 Presentation-II Query

Insert the exact query given during Presentation-II here before final submission. This is intentionally marked so the report matches the faculty-provided presentation evidence.

8. UI Design and Screenshots

The application uses a responsive web interface built with HTML5, CSS3 and JavaScript. The latest design includes a modern sidebar, KPI cards, status badges, responsive tables, animated dashboard counters and grouped blood-stock cards.

10.1 Screenshots to Insert Dashboard

Navigation

Dashboard

Donors

Patients

Hospitals

Donations

Inventory

Blood

Dashboard

Blood Bank Donation and Issue Management System

Total Donors

50

Total Patients

8

Total Hospitals

5

Inventory Records

16

Donor Management

Navigation

Dashboard

Donors

Patients

Hospitals

Donations

Donor Management

View and manage registered blood donors

Add New Donor

Donor Code

Example: D051

Donor Name

Enter donor name

Age

Patient Management

Patient Management

Register and manage patients requiring blood

Add New Patient

Patient Code

Example: P009

Patient Name

Enter patient name

Age

Hospital Management

Hospital Management

Manage hospitals connected with the blood bank

Add New Hospital

Hospital Code

Example: H006

HPlease fill out this field.

Enter hospital name

Address

Enter address

INSERT FINAL HOSPITAL MANAGEMENT SCREENSHOT HERE

Blood Donation Management

Record and manage blood donations

Record New Donation

Donation Code

Example: DN017

Donor ID

Example: 1

Blood Group

Select Blood Group

Blood Inventory

Blood Inventory

Current blood stock available by blood group

A+

18,000 mL

120 blood units

• Available

A-

8,400 mL

56 blood units

• Available

B+

24,000 mL

160 blood units

B-

10,800 mL

72 blood units

9. Implementation Details

9.1 Technology Stack

Layer	Technology	Role
Frontend	HTML5, CSS3, JavaScript	Forms, tables, dashboard and interactions
Backend	Python + Flask	Routing and database operations
Database	MySQL	Persistent relational storage
Connectivity	mysql-connector-python	Application-to-MySQL connection
Configuration	python-dotenv	Environment variables
Development	Visual Studio Code	Development and debugging

9.2 Database Connectivity

```
def get_db_connection():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST"),
        port=int(os.getenv("DB_PORT", 3306)),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME")
    )
```

Credentials are kept in a .env file and should not be committed to a public repository.

9.3 Application Flow

Browser → Flask Route → MySQL Query → Result Set → Jinja Template → Browser. For write operations, a POST form sends data to the backend, the backend executes a parameterized SQL statement, commits the transaction and redirects to the relevant module.

9.4 DBMS Features Demonstrated

Primary keys, foreign keys, UNIQUE constraints, CHECK constraints, ENUM fields, joins, aggregation, views, triggers, CRUD operations, parameterized SQL and application–database connectivity.

10. Testing

10.1 Test Cases and Results

TC	Test	Expected Result	Status
TC01	Open dashboard	Dashboard loads with statistics	Pass
TC02	View donors	Donor list displays	Pass
TC03	Add donor	New donor inserted	Pass
TC04	Delete donor	Selected donor removed	Pass
TC05	View patients	Patient records display	Pass
TC06	View hospitals	Hospital records display	Pass
TC07	View donations	Donation records display with 150 mL	Pass
TC08	View inventory	Inventory grouped by blood group	Pass
TC09	View requests	Request records display	Pass

TC10	View issues	Issue records display	Pass
TC11	Donation quantity validation	Invalid quantity rejected	Pass
TC12	Blood issue trigger	Inventory/request status updates	Pass
TC13	Foreign-key integrity	Invalid reference rejected	Pass

10.2 Demonstration Dataset

Table	Records
blood_groups	8
donors	50
hospitals	5
patients	8
donations	16
blood_inventory	16
blood_requests	8
blood_issues	5

11. Conclusion and Future Enhancements

11.1 Conclusion

The Blood Bank Donation and Issue Management System demonstrates the application of DBMS principles to a practical healthcare-management problem. The system provides structured storage and retrieval of donor, patient, hospital, donation, inventory, request and issue information. Constraints, views and triggers improve data integrity and automate selected operations.

The Flask interface provides a usable front end while MySQL provides the persistent relational data layer. The project therefore combines theoretical DBMS concepts with a working web application.

11.2 Future Enhancements

Enhancement	Description
Authentication	Admin, hospital and staff login with role-based access.
Cloud Deployment	Host application and database for public access.
Automated Alerts	Low-stock and expiry notifications.
Advanced Analytics	Donation trends, demand forecasting and blood-group analytics.
Blood Compatibility	Automated compatibility/matching assistance.
Audit Logs	Track record creation, updates and issue operations.
API Integration	Secure APIs for mobile or hospital applications.
Backup and Recovery	Scheduled database backup and recovery.

References

1. MySQL Documentation – SQL statements, constraints, views and triggers.
2. Flask Documentation – Flask web application framework and routing.
3. Python Documentation – Python programming language and standard library.
4. MDN Web Docs – HTML, CSS and JavaScript references.
5. Database Management Systems course lectures and laboratory material.
6. MySQL Workbench – database design, SQL execution and administration.

12. Appendix – GitHub Repository Link

The final source code should be uploaded to a GitHub repository containing the Flask application, HTML templates, CSS/JavaScript assets, SQL schema/data scripts and a README file.

Repository	Value
Project	Blood Bank Donation and Issue Management System
GitHub URL	https://github.com/ghoshsoubhik622-rgb/blood-bank-management-system.git
Branch	main

End of Project Report